JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Interface CompletionStage<T>

All Known Implementing Classes:

CompletableFuture

---

public interface **CompletionStage<T>**

A stage of a possibly asynchronous computation, that performs an action or computes a value when another CompletionStage completes. A stage completes upon termination of its computation, but this may in turn trigger other dependent stages. The functionality defined in this interface takes only a few basic forms, which expand out to a larger set of methods to capture a range of usage styles:

- The computation performed by a stage may be expressed as a Function, Consumer, or Runnable (using methods with names including *apply*, *accept*, or *run*, respectively) depending on whether it requires arguments and/or produces results. For example, `stage.thenApply(x -> square(x)).thenAccept(x -> System.out.print(x)).thenRun(() -> System.out.println())`. An additional form (*compose*) applies functions of stages themselves, rather than their results.
- One stage's execution may be triggered by completion of a single stage, or both of two stages, or either of two stages. Dependencies on a single stage are arranged using methods with prefix *then*. Those triggered by completion of *both* of two stages may *combine* their results or effects, using correspondingly named methods. Those triggered by *either* of two stages make no guarantees about which of the results or effects are used for the dependent stage's computation.
- Dependencies among stages control the triggering of computations, but do not otherwise guarantee any particular ordering. Additionally, execution of a new stage's computations may be arranged in any of three ways: default execution, default asynchronous execution (using methods with suffix *async* that employ the stage's default asynchronous execution facility), or custom (via a supplied Executor). The execution properties of default and async

modes are specified by CompletionStage implementations, not this interface. Methods with explicit Executor arguments may have arbitrary execution properties, and might not even support concurrent execution, but are arranged for processing in a way that accommodates asynchrony.

- Two method forms support processing whether the triggering stage completed normally or exceptionally: Method `whenComplete` allows injection of an action regardless of outcome, otherwise preserving the outcome in its completion. Method `handle` additionally allows the stage to compute a replacement result that may enable further processing by other dependent stages. In all other cases, if a stage's computation terminates abruptly with an (unchecked) exception or error, then all dependent stages requiring its completion complete exceptionally as well, with a `CompletionException` holding the exception as its cause. If a stage is dependent on *both* of two stages, and both complete exceptionally, then the CompletionException may correspond to either one of these exceptions. If a stage is dependent on *either* of two others, and only one of them completes exceptionally, no guarantees are made about whether the dependent stage completes normally or exceptionally. In the case of method `whenComplete`, when the supplied action itself encounters an exception, then the stage exceptionally completes with this exception if not already completed exceptionally.

All methods adhere to the above triggering, execution, and exceptional completion specifications (which are not repeated in individual method specifications). Additionally, while arguments used to pass a completion result (that is, for parameters of type T) for methods accepting them may be null, passing a null value for any other parameter will result in a `NullPointerException` being thrown.

This interface does not define methods for initially creating, forcibly completing normally or exceptionally, probing completion status or results, or awaiting completion of a stage. Implementations of CompletionStage may provide means of achieving such effects, as appropriate. Method `toCompletableFuture()` enables interoperability among different implementations of this interface by providing a common conversion type.

Since:

1.8

Method Summary

Abstract Methods

Modifier and Type	Method	Description
`CompletionStage<Void>`	`acceptEither(CompletionStage<? extends T> other, Consumer<? super T> action)`	Returns a new CompletionStage that, when either this or the other given stage

		complete normally is executed with t corresponding result as argumen to the supplied action.
`CompletionStage<Void>`	`acceptEitherAsync(CompletionStage<? extends T> other, Consumer<? super T> action)`	Returns a new CompletionStage that, when either this or the other given stage complete normally is executed using this stage's defaul asynchronous execution facility, with the corresponding result as argumen to the supplied action.
`CompletionStage<Void>`	`acceptEitherAsync(CompletionStage<? extends T> other, Consumer<? super T> action, Executor executor)`	Returns a new CompletionStage that, when either this or the other given stage complete normally is executed using the supplied executor, with the corresponding result as argumen to the supplied function.
`<U> CompletionStage<U>`	`applyToEither(CompletionStage<? extends T> other, Function<? super T,U> fn)`	Returns a new CompletionStage that, when either this or the other given stage complete normally is executed with t corresponding result as argumen to the supplied function.

<U> **CompletionStage**<U>	**applyToEitherAsync**(**CompletionStage**<? extends **T**> other, **Function**<? super **T**,U> fn)	Returns a new CompletionStage that, when either this or the other given stage complete normally is executed using this stage's defaul asynchronous execution facility, with the corresponding result as argumen to the supplied function.
<U> **CompletionStage**<U>	**applyToEitherAsync**(**CompletionStage**<? extends **T**> other, **Function**<? super **T**,U> fn, **Executor** executor)	Returns a new CompletionStage that, when either this or the other given stage complete normally is executed using the supplied executor, with the corresponding result as argumen to the supplied function.
CompletionStage<**T**>	**exceptionally**(**Function**<**Throwable**,? extends **T**> fn)	Returns a new CompletionStage that, when this stage completes exceptionally, is executed with this stage's exception the argument to t supplied function.
<U> **CompletionStage**<U>	**handle**(**BiFunction**<? super **T**,**Throwable**,? extends U> fn)	Returns a new CompletionStage that, when this stage completes either normally or exceptionally, is executed with this stage's result and exception as

		arguments to the supplied function.
<U> **CompletionStage**<U>	**handleAsync**(**BiFunction**<? super **T**,**Throwable**,? extends U> fn)	Returns a new CompletionStage that, when this stage completes either normally or exceptionally, is executed using this stage's default asynchronous execution facility, with this stage's result and exception as arguments to the supplied function.
<U> **CompletionStage**<U>	**handleAsync**(**BiFunction**<? super **T**,**Throwable**,? extends U> fn, **Executor** executor)	Returns a new CompletionStage that, when this stage completes either normally or exceptionally, is executed using the supplied executor, with this stage's result and exception as arguments to the supplied function.
CompletionStage<**Void**>	**runAfterBoth**(**CompletionStage**<? > other, **Runnable** action)	Returns a new CompletionStage that, when this and the other given stage both complete normally executes the given action.
CompletionStage<**Void**>	**runAfterBothAsync**(**CompletionStage**<? > other, **Runnable** action)	Returns a new CompletionStage that, when this and the other given stage complete normally, executes the given action

		using this stage's default asynchronous execution facility.
CompletionStage<Void>	runAfterBothAsync(CompletionStage<?> other, Runnable action, Executor executor)	Returns a new CompletionStage that, when this an the other given stage complete normally, executes the given action using the supplied executor.
CompletionStage<Void>	runAfterEither(CompletionStage<?> other, Runnable action)	Returns a new CompletionStage that, when either this or the other given stage complete normally executes the given action.
CompletionStage<Void>	runAfterEitherAsync(CompletionStage<?> other, Runnable action)	Returns a new CompletionStage that, when either this or the other given stage complete normally executes the given action using this stage's default asynchronous execution facility.
CompletionStage<Void>	runAfterEitherAsync(CompletionStage<?> other, Runnable action, Executor executor)	Returns a new CompletionStage that, when either this or the other given stage complete normally executes the given action using the supplied executor.
CompletionStage<Void>	thenAccept(Consumer<? super T> action)	Returns a new CompletionStage that, when this stage completes

		normally, is executed with this stage's result as th argument to the supplied action.
CompletionStage<Void>	thenAcceptAsync(Consumer<? super T> action)	Returns a new CompletionStage that, when this stage completes normally, is executed using thi stage's default asynchronous execution facility, with this stage's result as the argument to the supplied action.
CompletionStage<Void>	thenAcceptAsync(Consumer<? super T> action, Executor executor)	Returns a new CompletionStage that, when this stage completes normally, is executed using the supplied Executor with this stage's result as the argument to the supplied action.
<U> CompletionStage<Void>	thenAcceptBoth(CompletionStage<? extends U> other, BiConsumer<? super T,? super U> action)	Returns a new CompletionStage that, when this an the other given stage both complete normally is executed with th two results as arguments to the supplied action.
<U> CompletionStage<Void>	thenAcceptBothAsync(CompletionStage<? extends U> other, BiConsumer<? super T,? super U> action)	Returns a new CompletionStage that, when this an the other given stage complete normally, is

		executed using thi stage's default asynchronous execution facility, with the two resul as arguments to th supplied action.
<U> **CompletionStage**<**Void**>	**thenAcceptBothAsync**(**CompletionStage**<? extends U> other, **BiConsumer**<? super **T**,? super U> action, **Executor** executor)	Returns a new CompletionStage that, when this an the other given stage complete normally, is executed using the supplied executor, with the two resul as arguments to th supplied function.
<U> **CompletionStage**<U>	**thenApply**(**Function**<? super **T**,? extends U> fn)	Returns a new CompletionStage that, when this stage completes normally, is executed with this stage's result as th argument to the supplied function.
<U> **CompletionStage**<U>	**thenApplyAsync**(**Function**<? super **T**,? extends U> fn)	Returns a new CompletionStage that, when this stage completes normally, is executed using thi stage's default asynchronous execution facility, with this stage's result as the argument to the supplied function.
<U> **CompletionStage**<U>	**thenApplyAsync**(**Function**<? super **T**,? extends U> fn, **Executor** executor)	Returns a new CompletionStage that, when this stage completes normally, is

		executed using th supplied Executor with this stage's result as the argument to the supplied function.
<U,V> **CompletionStage**<V>	**thenCombine**(**CompletionStage**<? extends U> other, **BiFunction**<? super **T**,? super U,? extends V> fn)	Returns a new CompletionStage that, when this an the other given stage both complete normally is executed with t two results as arguments to the supplied function.
<U,V> **CompletionStage**<V>	**thenCombineAsync**(**CompletionStage**<? extends U> other, **BiFunction**<? super **T**,? super U,? extends V> fn)	Returns a new CompletionStage that, when this an the other given stage complete normally, is executed using thi stage's default asynchronous execution facility, with the two resul as arguments to t supplied function.
<U,V> **CompletionStage**<V>	**thenCombineAsync**(**CompletionStage**<? extends U> other, **BiFunction**<? super **T**,? super U,? extends V> fn, **Executor** executor)	Returns a new CompletionStage that, when this an the other given stage complete normally, is executed using th supplied executor, with the two resul as arguments to t supplied function.
<U> **CompletionStage**<U>	**thenCompose**(**Function**<? super **T**,? extends **CompletionStage**<U>> fn)	Returns a new CompletionStage that, when this stage completes normally, is

		executed with this stage as the argument to the supplied function.
<U> **CompletionStage**<U>	**thenComposeAsync**(**Function**<? super **T**,? extends **CompletionStage**<U>> fn)	Returns a new CompletionStage that, when this stage completes normally, is executed using thi stage's default asynchronous execution facility, with this stage as the argument to th supplied function.
<U> **CompletionStage**<U>	**thenComposeAsync**(**Function**<? super **T**,? extends **CompletionStage**<U>> fn, **Executor** executor)	Returns a new CompletionStage that, when this stage completes normally, is executed using the supplied Executor with this stage's result as the argument to the supplied function.
CompletionStage<**Void**>	**thenRun**(**Runnable** action)	Returns a new CompletionStage that, when this stage completes normally, executes the given action.
CompletionStage<**Void**>	**thenRunAsync**(**Runnable** action)	Returns a new CompletionStage that, when this stage completes normally, executes the given action using this stage's default asynchronous execution facility.
CompletionStage<**Void**>	**thenRunAsync**(**Runnable** action, **Executor** executor)	Returns a new CompletionStage

		that, when this stage completes normally, executes the given action using the supplied Executor.
CompletableFuture<**T**>	**toCompletableFuture**()	Returns a **CompletableFutu** maintaining the same completion properties as this stage.
CompletionStage<**T**>	**whenComplete**(**BiConsumer**<? super **T**,? super **Throwable**> action)	Returns a new CompletionStage with the same result or exceptio as this stage, that executes the giver action when this stage completes.
CompletionStage<**T**>	**whenCompleteAsync**(**BiConsumer**<? super **T**,? super **Throwable**> action)	Returns a new CompletionStage with the same result or exceptio as this stage, that executes the giver action using this stage's default asynchronous execution facility when this stage completes.
CompletionStage<**T**>	**whenCompleteAsync**(**BiConsumer**<? super **T**,? super **Throwable**> action, **Executor** executor)	Returns a new CompletionStage with the same result or exceptio as this stage, that executes the giver action using the supplied Executor when this stage completes.

Method Detail

thenApply

```
<U> CompletionStage<U> thenApply(Function<? super T,? extends U> fn)
```

Returns a new CompletionStage that, when this stage completes normally, is executed with this stage's result as the argument to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.

Type Parameters:

`U` - the function's return type

Parameters:

`fn` - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

thenApplyAsync

```
<U> CompletionStage<U> thenApplyAsync(Function<? super T,? extends U> fn)
```

Returns a new CompletionStage that, when this stage completes normally, is executed using this stage's default asynchronous execution facility, with this stage's result as the argument to the supplied function. See the `CompletionStage` documentation for rules covering exceptional completion.

Type Parameters:

`U` - the function's return type

Parameters:

`fn` - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

thenApplyAsync

```
<U> CompletionStage<U> thenApplyAsync(Function<? super T,? extends U> fn,
                                      Executor executor)
```

Returns a new CompletionStage that, when this stage completes normally, is executed using the supplied Executor, with this stage's result as the

argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Type Parameters:

`U - the function's return type`

Parameters:

`fn - the function to use to compute the value of the returned CompletionStage`

`executor - the executor to use for asynchronous execution`

Returns:

`the new CompletionStage`

thenAccept

`CompletionStage<Void> thenAccept(Consumer<? super T> action)`

Returns a new CompletionStage that, when this stage completes normally, is executed with this stage's result as the argument to the supplied action. See the CompletionStage documentation for rules covering exceptional completion.

Parameters:

`action - the action to perform before completing the returned CompletionStage`

Returns:

`the new CompletionStage`

thenAcceptAsync

`CompletionStage<Void> thenAcceptAsync(Consumer<? super T> action)`

Returns a new CompletionStage that, when this stage completes normally, is executed using this stage's default asynchronous execution facility, with this stage's result as the argument to the supplied action. See the CompletionStage documentation for rules covering exceptional completion.

Parameters:

`action - the action to perform before completing the returned CompletionStage`

Returns:

`the new CompletionStage`

thenAcceptAsync

```
CompletionStage<Void> thenAcceptAsync(Consumer<? super T> action,
                                      Executor executor)
```

Returns a new CompletionStage that, when this stage completes normally, is executed using the supplied Executor, with this stage's result as the argument to the supplied action. See the CompletionStage documentation for rules covering exceptional completion.

Parameters:

`action - the action to perform before completing the returned CompletionStage`

`executor - the executor to use for asynchronous execution`

Returns:

`the new CompletionStage`

thenRun

```
CompletionStage<Void> thenRun(Runnable action)
```

Returns a new CompletionStage that, when this stage completes normally, executes the given action. See the CompletionStage documentation for rules covering exceptional completion.

Parameters:

`action - the action to perform before completing the returned CompletionStage`

Returns:

`the new CompletionStage`

thenRunAsync

```
CompletionStage<Void> thenRunAsync(Runnable action)
```

Returns a new CompletionStage that, when this stage completes normally, executes the given action using this stage's default asynchronous execution facility. See the CompletionStage documentation for rules covering exceptional completion.

Parameters:

`action - the action to perform before completing the returned CompletionStage`

Returns:

`the new CompletionStage`

thenRunAsync

```
CompletionStage<Void> thenRunAsync(Runnable action,
                                   Executor executor)
```

Returns a new CompletionStage that, when this stage completes normally, executes the given action using the supplied Executor. See the CompletionStage documentation for rules covering exceptional completion.

Parameters:

`action - the action to perform before completing the returned CompletionStage`

`executor - the executor to use for asynchronous execution`

Returns:

`the new CompletionStage`

thenCombine

```
<U,V> CompletionStage<V> thenCombine(CompletionStage<? extends U> other,
                                     BiFunction<? super T,? super U,? extends V>
```

Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed with the two results as arguments to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Type Parameters:

`U - the type of the other CompletionStage's result`

`V - the function's return type`

Parameters:

other - the other CompletionStage

fn - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

thenCombineAsync

```
<U,V> CompletionStage<V> thenCombineAsync(CompletionStage<? extends U> other,
                                          BiFunction<? super T,? super U,? extend
```

Returns a new CompletionStage that, when this and the other given stage complete normally, is executed using this stage's default asynchronous execution facility, with the two results as arguments to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Type Parameters:

U - the type of the other CompletionStage's result

V - the function's return type

Parameters:

other - the other CompletionStage

fn - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

thenCombineAsync

```
<U,V> CompletionStage<V> thenCombineAsync(CompletionStage<? extends U> other,
                                          BiFunction<? super T,? super U,? extend
                                          Executor executor)
```

Returns a new CompletionStage that, when this and the other given stage complete normally, is executed using the supplied executor, with the two results as arguments to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Type Parameters:

U - the type of the other CompletionStage's result

V - the function's return type

Parameters:

other - the other CompletionStage

fn - the function to use to compute the value of the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

thenAcceptBoth

```
<U> CompletionStage<Void> thenAcceptBoth(CompletionStage<? extends U> other,
                                         BiConsumer<? super T,? super U> action)
```

Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed with the two results as arguments to the supplied action. See the CompletionStage documentation for rules covering exceptional completion.

Type Parameters:

U - the type of the other CompletionStage's result

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

thenAcceptBothAsync

```
<U> CompletionStage<Void> thenAcceptBothAsync(CompletionStage<? extends U> other,
                                              BiConsumer<? super T,? super U> act
```

Returns a new CompletionStage that, when this and the other given stage complete normally, is executed using this stage's default asynchronous execution facility, with the two results as arguments to the supplied action.

Type Parameters:

U - the type of the other CompletionStage's result

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

thenAcceptBothAsync

```
<U> CompletionStage<Void> thenAcceptBothAsync(CompletionStage<? extends U> other,
                                              BiConsumer<? super T,? super U> act
                                              Executor executor)
```

Returns a new CompletionStage that, when this and the other given stage complete normally, is executed using the supplied executor, with the two results as arguments to the supplied function.

Type Parameters:

`U - the type of the other CompletionStage's result`

Parameters:

`other - the other CompletionStage`

`action - the action to perform before completing the returned CompletionStage`

`executor - the executor to use for asynchronous execution`

Returns:

`the new CompletionStage`

runAfterBoth

```
CompletionStage<Void> runAfterBoth(CompletionStage<?> other,
                                   Runnable action)
```

Returns a new CompletionStage that, when this and the other given stage both complete normally, executes the given action. See the CompletionStage documentation for rules covering exceptional completion.

Parameters:

`other - the other CompletionStage`

`action - the action to perform before completing the returned CompletionStage`

Returns:

`the new CompletionStage`

runAfterBothAsync

```
CompletionStage<Void> runAfterBothAsync(CompletionStage<?> other,
                                        Runnable action)
```

Returns a new CompletionStage that, when this and the other given stage complete normally, executes the given action using this stage's default

asynchronous execution facility. See the CompletionStage documentation for rules covering exceptional completion.

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

runAfterBothAsync

```
CompletionStage<Void> runAfterBothAsync(CompletionStage<?> other,
                                        Runnable action,
                                        Executor executor)
```

Returns a new CompletionStage that, when this and the other given stage complete normally, executes the given action using the supplied executor. See the CompletionStage documentation for rules covering exceptional completion.

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

applyToEither

```
<U> CompletionStage<U> applyToEither(CompletionStage<? extends T> other,
                                     Function<? super T,U> fn)
```

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed with the corresponding result as argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Type Parameters:

U - the function's return type

Parameters:

other - the other CompletionStage

fn - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

applyToEitherAsync

```
<U> CompletionStage<U> applyToEitherAsync(CompletionStage<? extends T> other,
                                          Function<? super T,U> fn)
```

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using this stage's default asynchronous execution facility, with the corresponding result as argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Type Parameters:

U - the function's return type

Parameters:

other - the other CompletionStage

fn - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

applyToEitherAsync

```
<U> CompletionStage<U> applyToEitherAsync(CompletionStage<? extends T> other,
                                          Function<? super T,U> fn,
                                          Executor executor)
```

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using the supplied executor, with the corresponding result as argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Type Parameters:

U - the function's return type

Parameters:

other - the other CompletionStage

fn - the function to use to compute the value of the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

acceptEither

```
CompletionStage<Void> acceptEither(CompletionStage<? extends T> other,
                                   Consumer<? super T> action)
```

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed with the corresponding result as argument to the supplied action. See the CompletionStage documentation for rules covering exceptional completion.

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

acceptEitherAsync

```
CompletionStage<Void> acceptEitherAsync(CompletionStage<? extends T> other,
                                        Consumer<? super T> action)
```

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using this stage's default asynchronous execution facility, with the corresponding result as argument to the supplied action. See the CompletionStage documentation for rules covering exceptional completion.

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

acceptEitherAsync

```
CompletionStage<Void> acceptEitherAsync(CompletionStage<? extends T> other,
                                        Consumer<? super T> action,
                                        Executor executor)
```

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using the supplied executor, with the corresponding result as argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

runAfterEither

```
CompletionStage<Void> runAfterEither(CompletionStage<?> other,
                                     Runnable action)
```

Returns a new CompletionStage that, when either this or the other given stage complete normally, executes the given action. See the CompletionStage documentation for rules covering exceptional completion.

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

runAfterEitherAsync

```
CompletionStage<Void> runAfterEitherAsync(CompletionStage<?> other,
                                          Runnable action)
```

Returns a new CompletionStage that, when either this or the other given stage complete normally, executes the given action using this stage's default asynchronous execution facility. See the CompletionStage documentation for rules covering exceptional completion.

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

runAfterEitherAsync

```
CompletionStage<Void> runAfterEitherAsync(CompletionStage<?> other,
                                          Runnable action,
                                          Executor executor)
```

Returns a new CompletionStage that, when either this or the other given stage complete normally, executes the given action using the supplied executor. See the CompletionStage documentation for rules covering exceptional completion.

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

thenCompose

```
<U> CompletionStage<U> thenCompose(Function<? super T,? extends CompletionStage<U
```

Returns a new CompletionStage that, when this stage completes normally, is executed with this stage as the argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Type Parameters:

U - the type of the returned CompletionStage's result

Parameters:

fn - the function returning a new CompletionStage

Returns:

the CompletionStage

thenComposeAsync

```
<U> CompletionStage<U> thenComposeAsync(Function<? super T,? extends CompletionSt
```

Returns a new CompletionStage that, when this stage completes normally, is executed using this stage's default asynchronous execution facility, with this

stage as the argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Type Parameters:

U - the type of the returned CompletionStage's result

Parameters:

fn - the function returning a new CompletionStage

Returns:

the CompletionStage

thenComposeAsync

```
<U> CompletionStage<U> thenComposeAsync(Function<? super T,? extends CompletionSt
                                        Executor executor)
```

Returns a new CompletionStage that, when this stage completes normally, is executed using the supplied Executor, with this stage's result as the argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Type Parameters:

U - the type of the returned CompletionStage's result

Parameters:

fn - the function returning a new CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the CompletionStage

exceptionally

```
CompletionStage<T> exceptionally(Function<Throwable,? extends T> fn)
```

Returns a new CompletionStage that, when this stage completes exceptionally, is executed with this stage's exception as the argument to the supplied function. Otherwise, if this stage completes normally, then the returned stage also completes normally with the same value.

Parameters:

fn - the function to use to compute the value of the returned CompletionStage if this CompletionStage completed exceptionally

Returns:

the new CompletionStage

whenComplete

```
CompletionStage<T> whenComplete(BiConsumer<? super T,? super Throwable> action)
```

Returns a new CompletionStage with the same result or exception as this stage, that executes the given action when this stage completes.

When this stage is complete, the given action is invoked with the result (or `null` if none) and the exception (or `null` if none) of this stage as arguments. The returned stage is completed when the action returns. If the supplied action itself encounters an exception, then the returned stage exceptionally completes with this exception unless this stage also completed exceptionally.

Parameters:

`action - the action to perform`

Returns:

`the new CompletionStage`

whenCompleteAsync

```
CompletionStage<T> whenCompleteAsync(BiConsumer<? super T,? super Throwable> acti
```

Returns a new CompletionStage with the same result or exception as this stage, that executes the given action using this stage's default asynchronous execution facility when this stage completes.

When this stage is complete, the given action is invoked with the result (or `null` if none) and the exception (or `null` if none) of this stage as arguments. The returned stage is completed when the action returns. If the supplied action itself encounters an exception, then the returned stage exceptionally completes with this exception unless this stage also completed exceptionally.

Parameters:

`action - the action to perform`

Returns:

`the new CompletionStage`

whenCompleteAsync

```
CompletionStage<T> whenCompleteAsync(BiConsumer<? super T,? super Throwable> acti
                                     Executor executor)
```

Returns a new CompletionStage with the same result or exception as this stage, that executes the given action using the supplied Executor when this stage completes.

When this stage is complete, the given action is invoked with the result (or `null` if none) and the exception (or `null` if none) of this stage as arguments. The returned stage is completed when the action returns. If the supplied action itself encounters an exception, then the returned stage exceptionally completes with this exception unless this stage also completed exceptionally.

Parameters:

`action - the action to perform`

`executor - the executor to use for asynchronous execution`

Returns:

`the new CompletionStage`

handle

```
<U> CompletionStage<U> handle(BiFunction<? super T,Throwable,? extends U> fn)
```

Returns a new CompletionStage that, when this stage completes either normally or exceptionally, is executed with this stage's result and exception as arguments to the supplied function.

When this stage is complete, the given function is invoked with the result (or `null` if none) and the exception (or `null` if none) of this stage as arguments, and the function's result is used to complete the returned stage.

Type Parameters:

`U - the function's return type`

Parameters:

`fn - the function to use to compute the value of the returned CompletionStage`

Returns:

`the new CompletionStage`

handleAsync

```
<U> CompletionStage<U> handleAsync(BiFunction<? super T,Throwable,? extends U> fn
```

Returns a new CompletionStage that, when this stage completes either normally or exceptionally, is executed using this stage's default asynchronous execution facility, with this stage's result and exception as arguments to the supplied function.

When this stage is complete, the given function is invoked with the result (or `null` if none) and the exception (or `null` if none) of this stage as arguments, and the function's result is used to complete the returned stage.

Type Parameters:

U - the function's return type

Parameters:

fn - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

handleAsync

```
<U> CompletionStage<U> handleAsync(BiFunction<? super T,Throwable,? extends U> fn
                                   Executor executor)
```

Returns a new CompletionStage that, when this stage completes either normally or exceptionally, is executed using the supplied executor, with this stage's result and exception as arguments to the supplied function.

When this stage is complete, the given function is invoked with the result (or `null` if none) and the exception (or `null` if none) of this stage as arguments, and the function's result is used to complete the returned stage.

Type Parameters:

U - the function's return type

Parameters:

fn - the function to use to compute the value of the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

toCompletableFuture

```
CompletableFuture<T> toCompletableFuture()
```

Returns a `CompletableFuture` maintaining the same completion properties as this stage. If this stage is already a CompletableFuture, this method may return this stage itself. Otherwise, invocation of this method may be equivalent in effect to `thenApply(x -> x)`, but returning an instance of type `CompletableFuture`. A CompletionStage implementation that does not choose to interoperate with others may throw `UnsupportedOperationException`.

Returns:

the CompletableFuture

Throws:

`UnsupportedOperationException` - if this implementation does not interoperate with CompletableFuture

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.